

VAPI Developer SDK Integration Guide

*Client SDK API, Server Verification, Tournament Gating, and Privacy-Preserving Data
Flows*

Version 1.0 — Phase 228

April 2026

Author: VAPI Protocol Engineering

CONFIDENTIAL — ENGINEERING DISTRIBUTION

1. Document Purpose and Scope

This guide provides comprehensive, engineering-grade documentation for integrating the VAPI (Verified Autonomous Physical Intelligence) protocol into games, backend servers, and tournament infrastructure. It is the canonical reference for any team implementing VAPI attestation, from initial SDK initialization through live tournament adjudication.

The document covers:

- **Client-side SDK** — Device discovery, PoAC stream subscription, inference code handling, and humanity probability queries for Unity (C#) and Unreal Engine (C++).
- **Server-side verification** — On-chain eligibility checks, ECDSA-P256 signature verification, ZK proof validation, and session integrity pipelines.
- **Tournament gating** — Pre-match eligibility, live session monitoring, post-match adjudication, and the 10-condition preflight check.

- **UX patterns** — Device tier badges, error states, and privacy-respecting player-facing displays.
- **Privacy-preserving data flows** — Seven biometric privacy primitives (BP-001 through BP-007), ZK attestation, and regulatory compliance.
- **Telemetry and advisory handling** — L4 biometric anomaly scoring, transport telemetry, and advisory code processing.

Target integrators: Game client teams (C#/Unity, C++/Unreal), backend teams (any language with HTTP/WebSocket and Ethereum ABI support), and tournament operations personnel.

Reference device: Sony DualShock Edge (CFI-ZCP1) — Attested tier reference hardware.

Protocol phase: 228, with 43 live contracts on IoTeX Testnet (chain ID 4690).

2. Architecture Overview

2.1 End-to-End Data Flow

The VAPI protocol establishes a cryptographically verifiable chain of trust from the physical controller to the game server. The flow operates as follows:

Stage	Component	Function
1. Input	Controller (DualShock Edge)	Generates raw HID sensor data: buttons, sticks, touchpad, IMU (gyroscope + accelerometer), and optional GSR
2. Analysis	Bridge Service (Python asyncio, FastAPI)	Reads HID at 1,000 Hz via USB, runs 9 PITL detection layers in real time, assembles 228-byte PoAC records, signs with ECDSA-P256, chains via SHA-256(raw[0:164])
3. Anchoring	IoTeX L1 (43 contracts)	Receives PoAC anchors,

Stage	Component	Function
		ZK proofs, rulings, credential state. All on-chain data is publicly verifiable.
4. Verification	Game Server (SDK query)	Queries on-chain state via <code>VAPIProtocolLens.isFullyEligible(deviceId)</code> — a single view call, zero gas cost

2.2 Trust Tiers

VAPI defines three trust tiers. Each tier corresponds to a level of physical attestation confidence:

Tier	PITL Layers	Transport	Biometric Stack	Tournament Eligible
Attested	L0-L6 (all layers active)	1,000 Hz USB required	Full — PHCI-certified controller, soulbound VHP credential	Yes — all tournament tiers
Standard	L0-L5 minimum	250 Hz BLE acceptable	Partial — self-registered controller	Yes — standard tournaments only
Fallback / Unverified	L0-L3 only	Any	None	No — advisory analytics only, never for anti-cheat

IMPORTANT

Fallback-tier devices must never

be used as the basis for tournament anti-cheat decisions. They provide advisory analytics only

—

useful for casual play telemetry but not for competitive integrity enforcement.

3. Client SDK API Surface

3.1 SDK Initialization

The VAPI SDK is initialized once per application lifetime. It establishes a WebSocket connection to the local bridge service and configures the IoTeX RPC endpoint for on-chain queries.

C# (Unity)

```
using VAPI.SDK; public class VAPIManager : MonoBehaviour { private
VAPIClient _client; void Awake() { var config = new
VAPIClientConfig { BridgeEndpoint =
"ws://localhost:8080/ws/poac", IoTeXRpcUrl = "https://babel-
api.testnet.iotex.io", ProtocolLensAddress = "0x...", //
VAPIProtocolLens contract TimeoutMs = 5000,
TransportPreference = TransportType.USB_1000Hz }; _client =
new VAPIClient(config); _client.OnDeviceStatusChanged +=
HandleDeviceStatus; _client.OnPoACReceived += HandlePoAC;
_client.OnTierChanged += HandleTierChange;
_client.Initialize(); } void OnDestroy()
{ _client?.Dispose(); } private void
HandleDeviceStatus(DeviceStatusEvent e) { Debug.Log($"Device
{e.DeviceId}: {e.Status}"); } private void HandlePoAC(PoACRecord
record) { // Process incoming PoAC records if
(record.InferenceCode == 0x28 || record.InferenceCode == 0x29 ||
record.InferenceCode == 0x2A) { Debug.LogError("Hard
cheat detected – session compromised"); } } private void
HandleTierChange(TierChangeEvent e) { Debug.Log($"Tier changed:
{e.PreviousTier} -> {e.NewTier}"); } }
```

C++ (Unreal Engine)

```
#include "VAPI/VAPIClient.h" void
UVAPISubsystem::Initialize(FSubsystemCollectionBase& Collection)
{ Super::Initialize(Collection); FVAPIClientConfig Config;
Config.BridgeEndpoint = TEXT("ws://localhost:8080/ws/poac");
Config.IoTeXRpcUrl = TEXT("https://babel-api.testnet.iotex.io");
Config.ProtocolLensAddress = TEXT("0x..."); Config.TimeoutMs = 5000;
Config.TransportPreference = EVAPITransport::USB_1000Hz; Client =
MakeShared<FVAPIClient>(Config); Client-
```

```

>OnDeviceStatusChanged.AddUObject(          this,
&UVAPISubsystem::HandleDeviceStatus);      Client->OnPoACReceived.AddUObject(
this, &UVAPISubsystem::HandlePoAC);        Client->Initialize(); } void
UVAPISubsystem::Deinitialize() { if (Client.IsValid())
{      Client->Shutdown();      Client.Reset();      }
Super::Deinitialize(); }

```

3.2 Device Discovery and Status

Call `VAPIClient.GetConnectedDevices()` to retrieve all detected controllers. Each device exposes the following properties:

Property	Type	Description
<code>DeviceId</code>	<code>byte[32]</code>	keccak256(pubkey) — unique device identity
<code>Tier</code>	<code>enum</code>	Attested, Standard, or Fallback
<code>TransportType</code>	<code>enum</code>	USB_1000Hz or BLE_250Hz
<code>ControllerProfile</code>	<code>string</code>	e.g., "dualsense_edge", "dualsense", "xbox_elite_v2"
<code>FirmwareVersion</code>	<code>string</code>	Controller firmware semver
<code>IsEnrolled</code>	<code>bool</code>	Whether the device has completed biometric enrollment
<code>CredentialExpiry</code>	<code>DateTime</code>	Expiration of the soulbound VHP credential

C# — Querying Device Status

```

var devices = _client.GetConnectedDevices(); foreach (var device in devices)
{      Debug.Log($"ID: {device.DeviceId.ToHex()}");      Debug.Log($"Tier:
{device.Tier}");      Debug.Log($"Transport: {device.TransportType}");
Debug.Log($"Enrolled: {device.IsEnrolled}");      Debug.Log($"Credential
Expires: {device.CredentialExpiry}");      if (device.Tier ==
DeviceTier.Attested)      {      // Full tournament eligibility
EnableTournamentMode(device);      } }

```

3.3 PoAC Stream Subscription

The bridge service streams 228-byte Proof of Autonomous Cognition (PoAC) records over WebSocket in real time. Each record is cryptographically signed and hash-chained to the previous record, forming an immutable per-session attestation chain.

3.3.1 PoAC Record Structure (228 Bytes)

Offset	Length	Field	Type	Description
0	8	TimestampNs	uint64 BE	Unix epoch nanoseconds
8	1	InferenceCode	uint8	Detection result (see Section 3.4)
9	1	ConfidenceByte	uint8	0–255 maps to 0.0–1.0
10	4	SessionId	uint32 BE	Current session identifier
14	4	SequenceNumber	uint32 BE	Monotonically incrementing per session
18	2	FeatureDim	uint16 BE	Currently 13
20	2	<i>Reserved</i>	—	Padding (zero-filled)
22	8	AnomalyScore	float64 BE	L4 Mahalanobis anomaly score
30	8	ContinuityScore	float64 BE	L5 temporal continuity score
38	96	Features	12 × float64 BE	L4 feature vector (12 active features)
134	16	DeviceId	bytes	keccak256(pubkey) truncated to 128 bits
150	14	<i>Reserved</i>	—	Future use

Offset	Length	Field	Type	Description
				(zero-filled)
164	64	Signature	ECDSA-P256 r s	32-byte r concatenated with 32-byte s

CRITICAL — HASH DOMAIN

record_hash = SHA-256(raw[0:164])

—

the body ONLY, never the full 228 bytes. This is a

frozen protocol invariant

. The signature (bytes 164

—

227) is excluded from the hash. Separately,

DeviceId = keccak256(pubkey)

uses a different hash function. Do not conflate these two hash domains.

C# — PoAC Deserialization

```
public class PoACRecord {
    public ulong TimestampNs; // [0:8]
    uint64 big-endian public byte InferenceCode; // [8]
    0x00=NOMINAL, etc. public byte ConfidenceByte; // [9] 0-255 maps
    to 0.0-1.0 public uint SessionId; // [10:14] uint32 big-endian
    public uint SequenceNumber; // [14:18] uint32 big-endian public
    ushort FeatureDim; // [18:20] uint16 public double AnomalyScore;
    // [22:30] IEEE 754 double BE public double ContinuityScore; // [30:38]
    IEEE 754 double BE public double[] Features; // [38:134] 12x IEEE
    754 double BE public byte[] DeviceId; // [134:150] keccak256
    truncated public byte[] Signature; // [164:228] ECDSA-P256 r||s
    private byte[] _raw; public byte[] Body => _raw[0..164]; public
    float Confidence => ConfidenceByte / 255f; public string RecordHash
    {
        get { using var sha = SHA256.Create();
        return sha.ComputeHash(Body).ToHex(); } } public static
    PoACRecord Deserialize(byte[] raw) { if (raw.Length != 228)
    throw new VAPIFormatException($"PoAC must be exactly 228
```

```

bytes, got {raw.Length}");
var record = new PoACRecord { _raw =
raw };
record.TimestampNs =
BinaryPrimitives.ReadUInt64BigEndian(raw.AsSpan(0, 8));
record.InferenceCode = raw[8];
record.ConfidenceByte = raw[9];
record.SessionId =
BinaryPrimitives.ReadUInt32BigEndian(raw.AsSpan(10, 4));
record.SequenceNumber =
BinaryPrimitives.ReadUInt32BigEndian(raw.AsSpan(14, 4));
record.FeatureDim =
BinaryPrimitives.ReadUInt16BigEndian(raw.AsSpan(18, 2));
record.AnomalyScore =
BitConverter.Int64BitsToDouble(
BinaryPrimitives.ReadInt64BigEndian(raw.AsSpan(22, 8)));
record.ContinuityScore =
BitConverter.Int64BitsToDouble(
BinaryPrimitives.ReadInt64BigEndian(raw.AsSpan(30, 8)));
record.Features = new double[12];
for (int i = 0; i < 12; i++)
{
    int offset = 38 + (i * 8);
    record.Features[i] =
    BitConverter.Int64BitsToDouble(
        BinaryPrimitives.ReadInt64BigEndian(
            raw.AsSpan(offset, 8)));
}
record.DeviceId = raw[134..150];
record.Signature = raw[164..228];
return record;
} }

```

3.4 Inference Code Reference

Each PoAC record contains a single-byte inference code indicating the detection result from the 9 PITL layers. Codes are classified as *structural*, *hard cheat*, or *advisory*.

Code	Name	Type	Meaning	Tournament Impact
0x00	NOMINAL	Structural	Normal human input detected	No impact — eligible
0x28	DRIVER_INJECTION	Hard cheat	Hardware injection device detected (Cronus Zen, Titan One)	BLOCKS unconditionally
0x29	WALLHACK	Hard cheat	Behavioral wallhack signature detected	BLOCKS unconditionally
0x2A	AIMBOT	Hard cheat	Behavioral aimbot signature detected	BLOCKS unconditionally
0x2B	TEMPORAL_BO	Advisory	Timing	Contributes to

Code	Name	Type	Meaning	Tournament Impact
	T		quantization detected (macro scripts)	humanity_probability
0x30	BIOMETRIC_ANOMALY	Advisory	L4 Mahalanobis anomaly threshold exceeded	Contributes to humanity_probability
0x31	IMU_PRESS_DECOUPLED	Advisory	Button press without preceding IMU micro-movement	Contributes to humanity_probability
0x32	STICK_IMU_DECOUPLED	Advisory	Stick movement without correlating IMU orientation	Contributes to humanity_probability
0x33	GSR_CORRELATION_ABSENT	Advisory	No galvanic skin response correlation with game events	Future — GSR hardware required

SECURITY WARNING

Hard codes

{0x28, 0x29, 0x2A}

block tournament eligibility

unconditionally

. No override is possible at any integration level. If a single PoAC record in a session contains a hard cheat code, the entire session is tainted. Advisory codes feed the weighted humanity_probability formula and must never independently trigger player

blocks.

3.5 Humanity Probability API

The humanity probability is a weighted composite score derived from multiple PITL layer outputs. It represents the protocol's confidence that the input originates from an unassisted human operator.

3.5.1 Weighted Formula (Default — Without L6)

```
humanity_probability = 0.28 * p_L4 + 0.27 * p_L5 + 0.20 * p_E4 + 0.15 * p_L2B + 0.10 * p_L2C
```

3.5.2 Weighted Formula (With L6 Active)

```
p_human = 0.23 * p_L4 + 0.22 * p_L5 + 0.15 * p_E4 + 0.15 * p_L6 + 0.15 * p_L2B + 0.10 * p_L2C
```

NOTE

p_L2C

resolves to a 0.5 neutral prior in dead-zone stick games (e.g., NCAA CFB 26) where stick dead-zone analysis is not applicable. The

humanity_prob

value is stored in

SQLite only

—

it is

never

written on-chain. This is an immutable privacy guarantee.

C# — Querying Humanity Probability

```
public async Task<double> GetHumanityProbability(string deviceId) {    var  
response = await _client.Bridge.QueryAsync(  
$"/api/v1/device/{deviceId}/humanity_probability");    return  
response.GetDouble("humanity_prob");    // Returns: 0.0 - 1.0    // Typical  
human range: 0.82 - 0.98    // Investigation threshold: < 0.65 }
```

4. Server-Side Verification Flow

SECURITY WARNING

Game servers must

never

trust client-side attestation alone. All eligibility decisions in tournament mode

must

be verified server-side via on-chain queries and cryptographic signature verification.

4.1 On-Chain Eligibility Check

The primary integration point for game servers is a single contract call:

`VAPIProtocolLens.isFullyEligible(deviceId)`. This is a `view` function — it costs zero gas and can be called at any frequency.

Behind this single call, the `VAPIProtocolLens` contract composites six verification checks:

1. iolD registration verified (device has a valid DID:io identity)
2. ZK session proof verified (Groth16 proof anchored and valid)
3. Biometric credential not expired (soulbound ERC-4671 VHP)
4. Ruling registry clean (no active BLOCK ruling)
5. PoAd anchor present (adjudication hash committed)

6. Dual-primitive gate satisfied

Solidity Interface

```
interface IVAPIProtocolLens {    function isFullyEligible(bytes32 deviceId)
external view returns (bool); } interface IVAPIDualPrimitiveGate
{    function isDualEligible(bytes32 deviceId)    external view returns
(bool);    // = isFullyEligible() AND isRecorded() }
```

C# Server-Side — Eligibility Check (Nethereum)

```
using Nethereum.Web3; public class VAPIEligibilityService {    private
const string PROTOCOL_LENS_ADDRESS =    "0x..."; // VAPIProtocolLens on
IoTeX Testnet    private const string ABI = "[{\"inputs\":
[\"name\": \"deviceId\", \"
+
\"type\": \"bytes32\"}], \"name\": \"isFullyEligible\", \"
+
\"outputs\": [{\"type\": \"bool\"}], \"
+
\"stateMutability\": \"view\", \"type\": \"function\"}]]";    private readonly
Web3 _web3;    public VAPIEligibilityService()    {    _web3 = new
Web3(    "https://babel-api.testnet.iotex.io");    }    public
async Task<bool> CheckEligibility(string deviceId)    {    var contract
= _web3.Eth.GetContract(    ABI, PROTOCOL_LENS_ADDRESS);    var
function = contract.GetFunction("isFullyEligible");    return await
function.CallAsync<bool>(    deviceId.HexToByteArray());    } }
```

4.2 PoAC Signature Verification

Each 228-byte PoAC record carries an ECDSA-P256 signature over the 164-byte body. Server-side verification ensures the record was produced by the claimed device and has not been tampered with.

C# — ECDSA-P256 Signature Verification

```
using System.Security.Cryptography; public class PoACVerifier {    public
bool VerifySignature(byte[] poac, byte[] devicePubKey)    {    if
(poac.Length != 228)    throw new ArgumentException("Invalid PoAC
length");    if (devicePubKey.Length != 64)    throw new
ArgumentException(    "Expected 64-byte uncompressed public key
(X||Y)");    byte[] body = poac[0..164];    byte[] sig =
poac[164..228];    using var ecdsa = ECDSA.Create(new ECPParameters
{    Curve = ECCurve.NamedCurves.nistP256,    Q = new
ECPoint    {    X = devicePubKey[0..32],
Y = devicePubKey[32..64]    }    });    return
ecdsa.VerifyData(    body, sig, HashAlgorithmName.SHA256);    }
public bool VerifyChainIntegrity(    PoACRecord current, PoACRecord
previous)    {    // 1. Verify hash linkage    using var sha =
SHA256.Create();    var expectedHash = sha.ComputeHash(previous.Body);
// Compare with chain link slot    // 2. Verify timestamp monotonicity
if (current.TimestampNs <= previous.TimestampNs)    return false;
// 3. Verify sequence number incrementing    if (current.SequenceNumber
!= previous.SequenceNumber + 1)    return false;    return
true;    } }
```

C++ — ECDSA-P256 Verification (OpenSSL)

```
#include <openssl/ecdsa.h> #include <openssl/sha.h> #include
<openssl/obj_mac.h> bool VerifyPoACSignature(    const uint8_t* poac,
size_t len,    const uint8_t* pubkey, size_t pkLen) {    if (len != 228 ||
pkLen != 64) return false;    // Hash the body (first 164 bytes)
uint8_t digest[SHA256_DIGEST_LENGTH];    SHA256(poac, 164, digest);    //
```

```

Construct ECDSA signature from r||s    ECDSA_SIG* sig = ECDSA_SIG_new();
BIGNUM* r = BN_bin2bn(poac + 164, 32, nullptr);    BIGNUM* s =
BN_bin2bn(poac + 196, 32, nullptr);    ECDSA_SIG_set0(sig, r, s);    //
Construct EC key from public key bytes    EC_KEY* key =
EC_KEY_new_by_curve_name(    NID_X9_62_prime256v1);    EC_GROUP* group
= EC_GROUP_new_by_curve_name(    NID_X9_62_prime256v1);    EC_POINT*
point = EC_POINT_new(group);    BIGNUM* x = BN_bin2bn(pubkey, 32, nullptr);
BIGNUM* y = BN_bin2bn(pubkey + 32, 32, nullptr);
EC_POINT_set_affine_coordinates(    group, point, x, y, nullptr);
EC_KEY_set_public_key(key, point);    int result = ECDSA_do_verify(
digest, SHA256_DIGEST_LENGTH, sig, key);    // Cleanup omitted for brevity
return result == 1; }

```

4.3 ZK Proof Verification

VAPI uses Groth16 zero-knowledge proofs on the BN254 curve for the PitlSessionProof circuit. Each proof attests to a valid PITL session without revealing the underlying biometric witness data.

Public signals (nPublic = 5):

#	Signal	Description
1	featureCommitment	Poseidon(8) (scaledFeatures[0..6], inferenceCodeFromBody) — ZK-friendly commitment to the feature subset
2	anomalyScore	L4 Mahalanobis distance score
3	anomalyThreshold	Calibrated threshold (currently 7.009)
4	sessionId	Links to on-chain session record
5	inferenceResult	Must match inference code in the PoAC body

Key contract addresses:

- **PitlSessionProofVerifier:** 0x07D3ca1548678410edC505406f022399920d4072
- **PITLSessionRegistryV2:** 0x8da0A497234C57914a46279A8F938C07D3Eb5f12
- **CeremonyRegistry:** 0x739B5f... (beacon block #41723255)

4.4 Session Validation Pipeline

The following pseudocode represents the complete server-side validation sequence. Every step is mandatory for tournament-grade sessions.

```
SESSION VALIDATION PIPELINE ===== 1. Receive session_id from game client 2. Extract deviceId from session metadata 3. Call VAPIProtocolLens.isFullyEligible(deviceId) -> MUST return true; false = REJECT 4. Fetch PoAC chain for session from bridge API GET /api/v1/session/{session_id}/poac_chain 5. Verify chain integrity: a. prev_hash linkage: each record's hash slot matches SHA-256(previous_record[0:164]) b. Timestamp monotonicity: each timestamp_ns > previous timestamp_ns c. Sequence continuity: each sequence_number == previous + 1 6. Verify ECDSA-P256 signatures on EVERY record using the device's registered public key 7. Scan for hard cheat codes: IF any record contains {0x28, 0x29, 0x2A} -> REJECT session immediately 8. Verify ZK proof via PITLSessionRegistryV2 Call: isSessionProofValid(sessionId) 9. Check PHGCredential expiry: Soulbound ERC-4671, verify expiresAt > now 10. Check RulingRegistry for active BLOCK rulings: Call: hasActiveBlock(deviceId) 11. Return composite eligibility result to matchmaking
```

5. Tournament Gating Logic

5.1 Pre-Match Gating

Before any tournament match begins, the platform must query `isFullyEligible(deviceId)` for every participant. If any player fails the check, the match start must be blocked and the failure reason surfaced through UX (see Section 7).

- **Attested-only tournaments:** Additionally verify PHCI certification on-chain via `VAPIHardwareCertRegistry`.
- **Mixed tournaments:** Accept Standard tier but surface the tier badge to all players for transparency.

5.2 Tournament Preflight (Operator Tool)

The `run_tournament_preflight` tool checks 10 P0 conditions atomically before match commit-activation. The gate is fail-closed: `overall_pass = false` blocks activation.

#	Condition	Check	Threshold
1	separation_ok	Separation ratio between enrolled identities	≥ 0.70
2	l4_ok	L4 thresholds calibrated and not stale	Non-stale calibration
3	gate_ok	VAPIProtocolLens circuit-breaker green	Breaker = GREEN
4	cert_ok	PHGCredential not suspended	Status = active
5	audit_ok	MPC ceremony integrity verified	Ceremony valid
6	dual_gate_warned	VAPIDualPrimitive Gate status	Warning if degraded
7	epoch_window_warned	Epoch freshness window	Warning if near expiry
8	ioswarm_warned	ioSwarm emulator vs. live status	Warning if emulated
9	biometric_ttl_ok	Credential not TTL-expired AND renewal chain exists	TTL valid
10	all_pairs_p0_ok	All player pairs have separation ≥ 1.0	≥ 1.0 pairwise

5.3 Live Session Monitoring

During a match, the tournament platform subscribes to the PoAC stream for all participants simultaneously.

- **Hard cheat detection is real-time.** If 0x28, 0x29, or 0x2A appears mid-match, the session must be flagged immediately.
- **Advisory codes accumulate** into `humanity_probability`. Declining humanity scores should be surfaced to tournament operators via a dashboard, not to players.

- **Epistemic consensus gate:** 0.65 threshold. When ioSwarm is enabled, the weighted formula is {0.35 ClassJ, 0.35 Supervisor, 0.15 Triage, 0.15 ioSwarm}. When ioSwarm is disabled, the formula is {0.40, 0.40, 0.20} (ClassJ, Supervisor, Triage).

5.4 Post-Match Adjudication

The SessionAdjudicator (Agent #1) produces rulings after match completion. Rulings are one of five types:

Ruling	Effect	Escalation
BLOCK	Suspends PHGCredential (24h default; 7d if warmup_attack_score > 0.7)	Recorded on-chain — final
FLAG	Session flagged for human review	FLAG×5 → HOLD
HOLD	Session held pending investigation	HOLD×2 → BLOCK
CERTIFY	Session certified as human-authentic	Positive attestation record
CLEAR	Session cleared, no anomalies	No further action

On-chain commitment: Rulings are recorded via `RulingRegistry.sol` with:

```
commitment_hash = SHA-256( verdict + sorted(evidence_hashes) +
attestation_hash_hex + struct.pack(">Q", ts_ns) )
```

The `RulingRegistry` enforces a `UNIQUE commitmentHash` constraint for anti-replay protection.

ioSwarm dual-quorum veto:

- ClassJ nodes: `BLOCK_QUORUM = 0.67`
- Triage nodes: `BLOCK_QUORUM = 0.67`
- VHP mint: `MINT_QUORUM = 0.80` — fail-CLOSED (no quorum = no mint)

6. Telemetry and Advisory Handling

6.1 Advisory Code Processing

Advisory codes are informational — they contribute to `humanity_probability` but must never independently trigger player blocks. The bridge's 36-agent fleet processes advisories through epistemic consensus. Developers should trust the final ruling, not individual signals.

- Log all advisory codes for analytics.
- Do NOT display advisory codes to players.
- Do NOT make matchmaking decisions based on individual advisory occurrences.
- Trust the composite `humanity_probability` score and the SessionAdjudicator's final ruling.

6.2 L4 Biometric Telemetry

Every PoAC record includes `AnomalyScore` and `ContinuityScore` from the L4 and L5 layers respectively.

Metric	Threshold	Derivation	Human False Positive Rate
Anomaly Score (L4)	7.009	mean + 3σ	~2.9%
Continuity Score (L5)	5.367	mean + 2σ	—

These thresholds are USB-calibrated (N=74). BLE transport requires separate calibration. The L4 feature vector has 13 dimensions (12 active for threshold calculation; `touchpad_spatial_entropy` is initialized to 0.0 during gameplay).

6.3 Transport Telemetry

Transport	Poll Rate	Jitter Tolerance	Tier Impact
USB	1,000 Hz	$\pm 50\mu\text{s}$	Required for Attested tier
BLE	250 Hz	$\pm 2\text{ms}$	Acceptable for Standard tier only

Report transport type in session metadata for server-side tier verification. A transport type mismatch between the claimed tier and the observed poll rate should trigger a session flag.

7. UX Patterns for Device Tiers

7.1 Attested Devices

Display a **green shield badge** labeled "**VAPI Attested**". L0-L6 all active, full biometric verification. Surface this badge in the lobby, match screen, and post-match results.

Example display text:  **Attested Controller — Full VAPI Protection**

7.2 Standard Devices

Display a **yellow/amber shield badge** labeled "**VAPI Standard**". L0-L5 active, partial biometric stack. Clearly differentiate from Attested so players understand the tier difference.

Example display text:  **Standard Controller — VAPI Verified**

7.3 Unverified / Fallback Devices

Display a **gray/empty badge** labeled "**Unverified**". L0-L3 only, no biometric stack, no tournament eligibility. May display advisory-only analytics if the game supports casual mode.

Example display text:  **No VAPI Credential**

7.4 Error and Disconnection States

State	Display	Player Impact
Bridge disconnected	"VAPI Offline"	Casual play continues; tournament eligibility lost
Credential expired	"Credential Expired — Re-enrollment Required"	No tournament play until re-enrollment
Hard cheat detected	"Session Integrity Violation"	Session invalidated

IMPORTANT — EVASION PREVENTION

Never display raw inference codes, feature vectors, biometric data, or detection specifics to the player. Revealing detection details enables evasion learning. Use only the generic messages listed above.

8. Privacy-Preserving Data Flows

8.1 Core Privacy Guarantees

The VAPI protocol enforces seven biometric privacy primitives (BP-001 through BP-007). These are immutable protocol guarantees — no integrator can bypass or weaken them.

Primitive	Name	Mechanism
BP-001	Temporal Biometric Decay	90-day half-life; automatic expiration at 180 days. Stale biometric data self-destructs.

Primitive	Name	Mechanism
BP-002	ZK-Attested Consent	Consent as Poseidon hash — never raw signature. Consent is verifiable without revealing the consenting identity.
BP-003	Differential Privacy	Laplacian noise injection, $\epsilon \leq 1.0$ per year budget. Prevents re-identification from aggregated queries.
BP-004	K-Anonymity	$K \geq 5$ cohorts before threshold activation. No individual can be distinguished from at least 4 others.
BP-005	Homomorphic Processing	Separation ratios computed on encrypted data (Paillier/CKKS). Raw values never exposed.
BP-006	Shamir Sharing	Biometric identity split across 16 agents (8-of-16 threshold). No single agent holds a complete identity.
BP-007	Ephemeral Sessions	Raw biometric data exists only in RAM (<code>mlock()</code>), secure erase on session end). Never written to disk.

8.2 Developer Data Access Matrix

Data Source	Accessible Data	Access Method
On-chain	<code>isFullyEligible()</code> result (bool), PoAC chain hashes, ZK proof verification, ruling history, credential status	IoTeX RPC view calls
Bridge API	PoAC stream (228-byte records), inference codes, anomaly/continuity	WebSocket / REST

Data Source	Accessible Data	Access Method
	scores, humanity_probability	
NOT accessible	Raw sensor data, individual biometric features in cleartext, L4 Mahalanobis centroid, calibration session data	—

Regulatory compliance: BP-001 through BP-007 collectively satisfy GDPR Art. 9, CCPA/CPRA, BIPA, EU AI Act, PIPL, and LGPD requirements for biometric data processing.

8.3 Data Sovereignty

Players can pledge data sovereignty via the DataSovereigntyRegistry contract at three tiers: MANUFACTURER, DEVELOPER, and GAMER. The VAPIDataMarketplace governs anonymized data access with a 70/30 revenue split (70% to device pool, 30% to treasury). All data flows are consent-gated via ZK-attested consent (BP-002).

9. Sample Server Endpoints

9.1 Match Eligibility Endpoint

POST /api/v1/match/eligibility

```
// Request {      "player_device_ids":
[
  "0xaabbccdd11223344aabbccdd11223344",
  "0x55667788aabbccdd55667788aabbccdd"
] } // Response - 200 OK
{
  "eligible": true,      "players": [      {      "device_id":
"0xaabbccdd11223344aabbccdd11223344",      "tier": "attested",
"credential_valid": true,      "credential_expires": "2026-07-
15T00:00:00Z",      "ruling_status": "CLEAR",
"is_fully_eligible": true      },      {      "device_id":
"0x55667788aabbccdd55667788aabbccdd",      "tier": "standard",
"credential_valid": true,      "credential_expires": "2026-09-
01T00:00:00Z",      "ruling_status": "CLEAR",
"is_fully_eligible": true      }      ] }
```

9.2 Session Verification Endpoint

POST /api/v1/session/verify

```
// Request {      "session_id": 12345,      "device_id":
"0xaabbccdd11223344aabbccdd11223344" } // Response – 200 OK
{      "verified": true,      "chain_integrity": true,      "signature_valid":
true,      "hard_cheat_detected": false,      "zk_proof_verified": true,
"humanity_probability": 0.92,      "record_count": 3600,
"duration_seconds": 3600 }
```

9.3 Tournament Preflight Endpoint

POST /api/v1/tournament/preflight

```
// Request {      "tournament_id": "T-2026-0418-APEX",
"player_device_ids": [      "0xaabbccdd11223344aabbccdd11223344",
"0x55667788aabbccdd55667788aabbccdd",
"0x99001122334455669900112233445566"      ] } // Response – 200 OK
{      "overall_pass": true,      "conditions": {      "separation_ok":
true,      "l4_ok": true,      "gate_ok": true,      "cert_ok":
true,      "audit_ok": true,      "dual_gate_warned": false,
"epoch_window_warned": false,      "ioswarm_warned": false,
"biometric_ttl_ok": true,      "all_pairs_p0_ok": true      },
"warnings": [],      "checked_at": "2026-04-18T19:28:00Z" }
```

10. JSON Schemas

10.1 PoAC Record Schema

```
{      "$schema": "https://json-schema.org/draft/2020-12/schema",      "title":
"PoACRecord",      "type": "object",      "properties":
{      "timestamp_ns": {      "type": "integer",
"description": "Unix epoch nanoseconds"      },      "inference_code":
{      "type": "integer",      "minimum": 0,
"maximum": 255      },      "confidence": {      "type":
"number",      "minimum": 0.0,      "maximum": 1.0      },
"session_id": { "type": "integer" },      "sequence_number": { "type":
"integer" },      "feature_dim": { "type": "integer", "const": 13 },
"anomaly_score": { "type": "number" },      "continuity_score": { "type":
"number" },      "features": {      "type": "array",
"items": { "type": "number" },      "minItems": 12,
"maxItems": 12      },      "device_id": {      "type":
"string",      "pattern": "^0x[0-9a-f]{32}$"      },
"signature_r": {      "type": "string",      "pattern": "^0x[0-
9a-f]{64}$"      },      "signature_s": {      "type": "string",
"pattern": "^0x[0-9a-f]{64}$"      },      "record_hash": {
"pattern": "^0x[0-9a-f]{64}$",
"type": "string",      "description": "SHA-256(raw[0:164])"      },
"required": [
"timestamp_ns", "inference_code", "confidence",      "session_id",
"sequence_number", "anomaly_score",      "continuity_score", "device_id",
"record_hash"      ] }
```

10.2 Device Capability Manifest Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "DeviceCapabilityManifest",
  "type": "object",
  "properties": {
    "device_id": { "type": "string" },
    "controller_profile": {
      "type": "string",
      "enum": [
        "dualsense_edge",
        "xbox_elite_v2", "xbox_standard",
        "switch_pro"
      ]
    },
    "tier": {
      "type": "string",
      "enum": ["attested", "standard", "fallback"]
    },
    "transport_type": {
      "type": "string",
      "enum": ["usb_1000hz", "ble_250hz"]
    },
    "pitl_layers": {
      "type": "object",
      "properties": {
        "L0": { "type": "boolean" },
        "L1": { "type": "boolean" },
        "L2": { "type": "boolean" },
        "L2B": { "type": "boolean" },
        "L2C": { "type": "boolean" },
        "L3": { "type": "boolean" },
        "L4": { "type": "boolean" },
        "L4_partial": { "type": "boolean" },
        "L5": { "type": "boolean" },
        "L6": { "type": "boolean" },
        "L6_partial": { "type": "boolean" }
      }
    },
    "firmware_version": { "type": "string" },
    "phci_certified": { "type": "boolean" },
    "credential_status": {
      "type": "string",
      "enum": ["active", "expired", "suspended", "none"]
    },
    "credential_expires_at": {
      "type": "string",
      "format": "date-time"
    },
    "enrollment_status": {
      "type": "string",
      "enum": ["enrolled", "pending", "not_enrolled"]
    },
    "calibration_sessions": {
      "type": "integer",
      "minimum": 0
    },
    "required": [
      "device_id", "controller_profile", "tier",
      "transport_type", "pitl_layers",
      "phci_certified",
      "credential_status"
    ]
  }
}
```

10.3 Tournament Eligibility Response Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "TournamentEligibilityResponse",
  "type": "object",
  "properties": {
    "overall_pass": { "type": "boolean" },
    "conditions": {
      "type": "object",
      "properties": {
        "separation_ok": { "type": "boolean" },
        "l4_ok": { "type": "boolean" },
        "gate_ok": { "type": "boolean" },
        "cert_ok": { "type": "boolean" },
        "audit_ok": { "type": "boolean" },
        "dual_gate_warned": { "type": "boolean" },
        "epoch_window_warned": { "type": "boolean" },
        "ioswarm_warned": { "type": "boolean" },
        "biometric_ttl_ok": { "type": "boolean" },
        "all_pairs_p0_ok": { "type": "boolean" }
      }
    },
    "required": [
      "separation_ok", "l4_ok", "gate_ok",
      "cert_ok", "audit_ok", "biometric_ttl_ok",
      "all_pairs_p0_ok"
    ],
    "warnings": {
      "type": "array",
      "items": {
        "type": "string",
        "checked_at": {
          "type": "string",
          "format": "date-time"
        }
      }
    },
    "required": ["overall_pass", "conditions", "checked_at"]
  }
}
```

11. Test Vectors

11.1 PoAC Deserialization Test Vector

The following 228-byte hex string represents a valid PoAC record. Use it to validate your deserialization implementation.

```
Hex (228 bytes): 0000018F3A7C1E00 // TimestampNs: 1713472577024 (uint64 BE)
00 // InferenceCode: 0x00 (NOMINAL) CC //
ConfidenceByte: 204 (= 0.80 confidence) 0000303A // SessionId: 12346
(uint32 BE) 00000001 // SequenceNumber: 1 (uint32 BE) 000D
// FeatureDim: 13 (uint16 BE) 0000 // Reserved (2 bytes)
4012E147AE147AE1 // AnomalyScore: 4.72 (float64 BE) 4009A3D70A3D70A4 //
ContinuityScore: 3.21 (float64 BE) [96 bytes] // Features: 12x float64
BE values [16 bytes] // DeviceId: keccak256(pubkey)[0:16] [14 bytes]
// Reserved [64 bytes] // Signature: ECDSA-P256 r(32B)||s(32B)
```

Expected parsed values:

- `TimestampNs`: 1713472577024
- `InferenceCode`: 0x00 (NOMINAL)
- `Confidence`: 0.80 (204/255)
- `SessionId`: 12346
- `SequenceNumber`: 1
- `AnomalyScore`: 4.72
- `ContinuityScore`: 3.21
- `record_hash`: SHA-256(raw[0:164]) — compute from your deserialized body

11.2 Signature Verification Test Vector

Use the following to validate your ECDSA-P256 verification implementation:

Component	Description
Public Key (X)	32-byte X coordinate of the P256 public key point
Public Key (Y)	32-byte Y coordinate of the P256 public key point

Component	Description
Message	raw[0:164] — the 164-byte PoAC body
Signature r	raw[164:196] — 32-byte r component
Signature s	raw[196:228] — 32-byte s component
Expected result	true — signature is valid for the body under the given public key

11.3 Chain Integrity Test Vector

Given two consecutive PoAC records (Record A and Record B), verify:

- Hash linkage:** The hash reference in Record B must equal `SHA-256(Record_A[0:164])`.
- Timestamp monotonicity:** `Record_B.TimestampNs > Record_A.TimestampNs`.
- Sequence increment:** `Record_B.SequenceNumber == Record_A.SequenceNumber + 1`.

11.4 Inference Code Test Vectors

Scenario	InferenceCode	Expected Behavior
Normal human input	0x00 (NOMINAL)	Eligible — no action required
Hardware injection detected	0x28 (DRIVER_INJECT)	BLOCKED — session rejected unconditionally
Biometric anomaly	0x30 (BIOMETRIC_ANOMALY)	Advisory only — feeds <code>humanity_probability</code> , no independent block
Aimbot behavior	0x2A (AIMBOT)	BLOCKED — session rejected unconditionally

12. Error Handling and Fail Modes

12.1 Bridge Disconnection

- If the bridge connection is lost, the game continues but the PoAC chain is broken.
- The chain gap is detected at the L1 structural layer.
- **Fail-open** for casual play — the player can continue but loses attestation.
- **Fail-closed** for tournament eligibility — the session is no longer tournament-valid.
- On reconnection, the chain resumes with a gap marker. The gap is recorded but does not invalidate prior records.

12.2 Contract Query Failure

- If IoTeX RPC is unavailable, cache the last-known eligibility result with a TTL of **5 minutes** (recommended).
- **Never assume eligible on query failure in tournament mode** — fail-closed.
- Retry with exponential backoff: 1s → 2s → 4s (max 3 retries).

12.3 Invalid PoAC Records

Condition	Action	Rationale
Signature verification failure	Discard record, log event, do NOT block player	Could be transport corruption
Chain integrity failure	Flag session for manual review	May indicate tampering or bridge restart
Duplicate sequence_number	Flag immediately — replay attack	Sequence numbers must be strictly monotonic

12.4 Agent-Specific Fail Modes

All 36 agents in the VAPI fleet have defined fail-open or fail-closed behavior. Developers should trust the final on-chain state, not individual agent signals.

Agent / Component	Fail Mode	Reasoning
ioSwarm MINT	Fail-CLOSED (MINT_QUORUM=0.80)	No quorum = no VHP mint. Safety-critical.
TournamentPreflightAgent	Fail-CLOSED	Any preflight failure blocks match activation.
SessionAdjudicator	Fail-open	Adjudication is reversible; CLEAR on error is recoverable via manual review.
ioSwarm Adjudication	Fail-open	CLEAR on no quorum. Ensures liveness over strict safety for non-mint operations.

13. Integration Checklist

10. **Register as VAPI developer** — obtain API key and SDK license from the VAPI Developer Portal.
11. **Install VAPI SDK** — NuGet package for C# (`VAPI.SDK`), `vcpkg` or Conan for C++ (`vapi-client`).
12. **Configure endpoints** — set bridge WebSocket endpoint and IoTEx RPC URL in the SDK config.
13. **Implement device discovery** — call `GetConnectedDevices()` and handle tier classification.
14. **Subscribe to PoAC stream** — register `OnPoACReceived` callback, implement 228-byte deserialization.
15. **Implement server-side eligibility** — call `VAPIProtocolLens.isFullyEligible(deviceId)` via IoTEx RPC.

16. **Implement signature verification** — ECDSA-P256 over raw[0:164] with the device's public key.
17. **Add inference code handling** — hard cheat blocking ({0x28, 0x29, 0x2A}) + advisory logging.
18. **Implement UX tier badges** — Attested (green), Standard (amber), Unverified (gray).
19. **Add tournament gating** — preflight (10 conditions) + live monitoring + post-match adjudication.
20. **Implement privacy-preserving data flows** — no raw biometrics, ZK verification, respect BP-001-BP-007.
21. **Run certification test suite** — execute the VAPI Certification Test Suite (separate document).
22. **Submit for integration certification** — VAPI Protocol Engineering reviews your implementation.
23. **Deploy to staging** — set dry_run=true, validate all flows end-to-end without affecting live players.
24. **Production deployment** — enable live mode, monitor telemetry dashboards for first 72 hours.

TIP

Steps 1

—

5 can be completed in parallel by client and server teams. Step 6 is the critical server-side integration point. Do not proceed to Step 10 (tournament gating) until

Steps 6

—

8 pass automated validation.

14. SDK Version Compatibility

Component	Value
Protocol Phase	228
SDK Version	Aligned to Phase 228
Bridge Test Count	2,192
SDK Test Count	418 (402 passing, 16 skipped — pending L6 hardware)
Contract Count	43 LIVE on IoTeX Testnet (chain ID 4690)
Agent Count	36 active
Tool Count	149
Reference Device	Sony DualShock Edge (CFI-ZCP1)
PoAC Record Size	228 bytes (frozen invariant)
PITL Layers	9 (L0-L6 + L2B + L2C)

15. Appendix: Contract Address Reference

All addresses below are on the IoTeX Testnet (chain ID 4690).

Contract	Address	Purpose
VAPIProtocolLens	0x... <i>(see deployment manifest)</i>	Composable isFullyEligible() gate
PITLSessionRegistryV2	0x8da0A497234C57914a46279A8F938C07D3Eb5f12	ZK session proof registry

Contract	Address	Purpose
PitlSessionProofVerifier	0x07D3ca1548678410edC505406f022399920d4072	Groth16 on-chain verifier
VAPIDualPrimitiveGate	0xd7b146...	isDualEligible() composite gate
RulingRegistry	0xa3A235...	Anti-replay ruling commitments
AdjudicationRegistry	0x44CF98...	PoAd hash registry
CeremonyRegistry	0x739B5f...	MPC ceremony audit trail
SeparationRatioRegistry	0xB39CeE...	Separation ratio commitments
VHPReenrollmentBadge	0x42E7A2...	Soulbound re-enrollment badge (ERC-4671)
DataSovereigntyRegistry	0xd928d9...	Player data sovereignty pledges
VAPIHardwareCertRegistry	<i>Deployment pending</i>	Hardware certification (PHCI)
VAPIToken	<i>Deployment pending</i>	ERC-20 protocol governance token

16. Glossary

Term	Definition
PoAC	Proof of Autonomous Cognition — 228-byte cryptographic attestation record proving that controller input originates from an unassisted human operator.
PITL	Physical Intelligence Trust Layer — 9 detection layers (L0-L6 plus L2B, L2C) that analyze controller input for human authenticity.
VHP	Verified Human Proof — a soulbound ERC-4671 biometric credential attesting to a verified human identity behind a device.
PHG	Player Human-Genome — credential registry associating biometric profiles

Term	Definition
	with device identities.
PHCI	Physical Hardware Certification Index — certification that a specific controller model meets VAPI hardware attestation requirements.
ioSwarm	Decentralized adjudication consensus network composed of ClassJ, Supervisor, and Triage nodes that produce distributed rulings.
Mahalanobis Distance	Statistical distance measure accounting for feature correlations, used in L4 biometric anomaly detection.
Groth16	Zero-knowledge proof system on the BN254 curve, used for PitlSessionProof circuit verification.
Poseidon	ZK-friendly hash function (8-input variant) used for in-circuit feature commitments.
ioID	IoTeX device identity standard (DID:io) providing decentralized device registration and key management.
DePIN	Decentralized Physical Infrastructure Network — the broader ecosystem category for protocols like VAPI that bridge physical devices to blockchain verification.
PoAd	Proof of Adjudication — on-chain hash anchor for a completed adjudication process.
ECDSA-P256	Elliptic Curve Digital Signature Algorithm on the NIST P-256 curve, used for PoAC record signing.

— End of Document —

VAPI Developer SDK Integration Guide — v1.0 — Phase 228 — April 2026

CONFIDENTIAL — Engineering Distribution